

Research Proposal

In this section, you must incorporate the following aspects:

- a) Theoretical-conceptual and state-of-the-art developments that underpin the proposal
- b) Objectives and hypotheses or research questions
- c) Methodology
- d) Work Plan
- e) Background information to assess the capacity and experience of the team to implement the proposal

Be sure to highlight the potential scientific or technological impact and novelty of your proposal. Keep in mind the provisions contained in the Guidelines for the Regular Fondecyt National Projects Competition 2021.

This file must contain a maximum of **10 pages** (use letter size format, Verdana font size 10 or similar).

Introduction

One of the key steps to solve scientific and engineering problems is the generation of a proper spatial discretization of a domain or object. Problems can be related to hydrological modeling, the design of a new device, fluid dynamics or fracture mechanics, among others. Two of the most studied discretization methods and their use as the basis of several meshing tools are the Delaunay partition and its dual the Voronoi diagram. Depending on the problem and the underlying numerical method, a Delaunay triangulation with the required element quality and point distribution is enough to get good results. This is the case for some problems solved with the Finite Element Method (FEM) where proper triangulations generated by Detri2 [1] or Triangle [2] in 2D, or Tetgen in 3D [3] are perfect. In case of problems solved with the Virtual Element Method (VEM), convex/non-convex polygons of any shape can be used [4, 5], and usually meshes based in the Voronoi diagram such as the generated by PolyMesher [6] are used. When problems are solved with the Control Volume Method (CVM), the combination of both, the Delaunay partition and the Voronoi diagram is an option. Proper mixed element Delaunay meshes for the CVM are described in [7]. Together with tetrahedra, the mesh includes cuboids, prisms, and pyramids, among others, as long as these elements are co-spherical.

In 2D(3D), a Delaunay partition of the point set P is a triangulation(tetrahedron mesh) if there are no co-circular(co-spherical) points. The Delaunay condition is fulfilled if the circumcircle(circumsphere) of each mesh cell does not include anyother point p of P in its interior. Among the good properties of a 2D Delaunay partition of a point set P is that it maximizes the size of its smallest angle over all the possible partitions for this point set P . In 3D this property does not hold. In its dual, the Voronoi diagram, each Voronoi region contains a point p in its interior and surrounds all the points of the space that are closer to p than any other point q , with $p, q \in P$. The Voronoi diagram can be generated from scratch or from a the Delaunay partition. The Voronoi points are the circumcenters of each Delaunay cell, the Voronoi edges in 2D (faces in 3D) are the bisectors of Delaunay edges. Voronoi regions are convex polygons in 2D (convex polyhedra in 3D).

Since the last eight years we have been working in the improvement of meshing algorithms for the generation of polygon and polyhedron meshes. In 2D, we have been working on (i) polygon meshes inspired in packing algorithms [8], (ii) polygon meshes for hydrological modeling [9, 10], (iii) the integration of a polyhedron Delaunay tessellation strategy [11] inside of a modified octree meshing tool [12] (extended Mesher), (iv) 2D Voronoi meshes and a VEM library (Delanoi and Veamy, respectively) [13], (iv) a framework to test and compare polygon meshing algorithms and (v) applying Delaunay triangulations for particle tracking [14] in 2D and for searching cosmic polygonal voids [15]. This research proposal is the natural continuation of this work but here we want to put emphasis in the generation of a new algorithm that can generate from the first step an initial polygon/polyhedron partition of the domain. The main difference with meshes based on the Voronoi diagram is that the initial cells are from the beginning inside the domain and polygons/polyhedra might be non-convex. We expect that for the same point density, these meshes will be formed by less edges and cells than meshes based on the Voronoi meshes, and so improve

the performance of numerical solvers that depend on the number of these edges and cells. We expect that this kind of meshes will benefit several applications in science and engineering, but in particular the Virtual Element Method (VEM) [16, 17] in order to provide an alternative spatial discretization, more easily to adapt to complex geometries and to refine mesh elements than meshes based on the Voronoi diagram. It is already known that the use of polygon/polyhedral cells in fracture mechanics simulations using the VEM facilitate the geometrical constraints that are originated by cracks that cut cells; once a crack cuts an cell, it can be subdivided into two or more cells that are not restricted to be triangles/tetrahedra or quadrilaterals/hexahedra of good quality; they can be arbitrary polygonal/polyhedral cells (convex or not) [18, 19].

To the best of our knowledge, since the need of proper polygon/polyhedron meshes is quite new, efficient algorithms to automatically generate appropriate polygon/polyhedron meshes that at the same time take care of reducing as much as possible the number of points and cells, have not been developed yet. As mentioned before, the most used meshes are meshes based on the Voronoi diagram because there are several algorithms and tools to generate and adapt them to the VEM but we believe there is still space to develop new algorithms that are more powerful than the current ones. In addition, we believe that the requirements of each specific numerical method and application can be properly encapsulated so that we can design meshing algorithms to develop a flexible mesh generator easily adaptable to new requirements.

This proposal is organized as follows: it starts with the motivation and challenges in the meshing field, in particular for polygon and polyhedron meshes. It continues with our approach included as part of the conceptual framework and research problem, and the related work. Then, it ends with the hypotheses, goals, methodology, work plan and work in progress.

Motivation

In this section we describe some applications that keep our interest in developing more efficient, scalable and robust algorithms for the generation of polygon/polyhedron meshes. These are the following:

- Simulation of hydrological processes: The requirements come from the numerical models used to simulate hydrological processes at urban and peri-urban scales. The initial mesh is obtained by intersecting polyline layers of land use, rivers, or sewers among other natural or relevant man-made features. Initial meshes composed of usually non convex polygons very bad shaped with holes must be improved in order to generate proper polygons, keeping low the number of points and polygons while preserving the main geometric features. A good quality polygon has its centroid inside it, a convexity index and form factor close to 1, among other characteristics. Our approach was to generate a triangulation that exactly fits the geometry (initial mesh), and then join triangles to form adequate convex/non-convex polygons [9, 10]. Currently we are exploring how to mix superficial hydrological and groundwater models to study the water resources in Chile.
- Fracture simulation in solid mechanics: The Virtual Element Method (vem) [16, 17] is a generalization of the finite element method that can be used with cells of arbitrary shape. The mathematical expressions for the linear system can be analytically solved; then the solutions are free of integration errors. Therefore, the vem has been used to simulate several problems in a more precise and efficient way than the pfem and standard numerical methods [20]. In particular, for fracture simulation, a good quality mesh must have regions with different levels of refinement according to what the simulation requires. It seems not to be required to avoid cells with too small or large angles, or bad aspect ratio, as finite elements need. It may be necessary to locally refine a coarse cell into polygons/polyhedra if the fracture goes through and add new points close to the crack tip in order to compute the stresses around it [21,19].
- Semiconductor device simulation: our group decided to approach this problem using mixed-element Delaunay meshes composed of cuboids, prisms, pyramids and tetrahedra and the Finite Volume Method(FVM)[22, 23]. We required both the edges of the Delaunay tessellation and the Voronoi regions as finite volumes in order to simplify the computation of the numerical integration. Our challenge is to automatically generate polyhedron Delaunay meshes for complex domains with the minimum number of points and cells that fulfil the point density requirements. In order to advance in this direction, in the last

two years we have integrated a strategy [11] that partition a convex element with Steiner points into co-spherical polyhedral elements into the mesh generator Mesher [12], a mixed element mesh generator based on modified octrees. This strategy was previously applied to generate polyhedron meshes inside convex domains. We are currently evaluating the impact in the point and element reduction in comparison with previous approaches. Then we plan to test it to solve problems with the VEM.

Polygonal and polyhedral meshes: the current challenges

There are several numerical methods, and each one has a range of applications where they give accurate results in a reasonable time, but there is still no method that can be used to solve any problem[18]. In this sense, the VEM, a quite new method, is being explored because it is allowing to solve problems such as finite strain elasto-plasticity and fracture mechanics, among others, in a robust and efficient way[18]. Wriggers et al. also state that the application range of VEMs is still not known and so it is still an open challenge to determine it. Since the mesh can be formed by any polygon, it reduces strongly the number of mesh elements in comparison with the number of elements of a quality triangulation, allowing to solve new problems that are not possible with the standard numerical methods.

Each time a problem can be solved, new requirements appear to model a more complex domain than before, to improve the simulation speed or the precision of current methods, among other challenges. There are some applications like climate modeling and geological phenomena[24] that need huge meshes to get acceptable simulation results. A first approach to manage huge meshes is parallelizing algorithms on multicore, clusters, and/or gpu architectures. A second approach is to model the problem (if it is possible) with a numerical method that allows one to use less cells and mesh points than before to solve the same problem. Both approaches are not contradictory, because the second one can also benefit from the first one when the problem size increases.

The possibility to count with polygon/polyhedron mesh generators, whose generated meshes represent complex domain geometries as good as triangle and tetrahedral meshes, but with much less points and cells and similar simulation results, belongs to the second approach. We plan to mainly work with the second approach and our challenge will be to design a method that can generate a partition of the domain into polygon/polyhedron in a simple way, more simple and efficient than methods based on Voronoi constrained meshes.

The main challenges for the meshing field independent of the cell type can be summarized as the need of more robust and efficient algorithms for the:

- Automatically generation of good quality meshes for complex domain geometries and size problems (solid mechanics, hydrological, geological and climate modeling, among others)
- Efficient handling of huge meshes through the simulation pipeline in order to implement and execute more complex and accurate models than now (gpu-computing, multicore, clusters)

Since the numerical methods that can use polygon/polyhedron meshes are quite new, the used meshing tools have been generally developed to show that these numerical methods can be more accurate and give better results than the standard methods. As we have mentioned before, the most used polygon/polyhedron meshes are Voronoi meshes [25, 26, 27, 6, 20, 19] and triangulations(tetrahedral meshes), whose triangles(tetrahedra) are divided into polygons (polyhedra) [28, 29]. Then, we believe that there are still several challenges in the generation of polygon/polyhedron meshes and these are:

- The need of efficient and robust algorithms to generate a good enough starting polygon/polyhedron mesh on complex domains reducing using as less cells as possible.
- Identification, design, and development of quality metrics for polygonal/polyhedral cells to guide the point insertion and mesh quality improvement process during mesh adaptation
- Design of strategies to efficiently divide polygons/polyhedra into new polygonal/polyhedral cells hopefully by adding as few points as possible while preserving the element quality

- Design of scalable polygon/polyhedron meshing algorithms through the development of parallel algorithms

These challenges have motivated us to elaborate this research proposal. We think there is still space to contribute with novel algorithms to build meshing tools that allow us to model more accurate and complex problems than before.

Conceptual framework and research questions

As we have told in the previous section, there is still no numerical method that can be used to solve any problem. The same occurs with the kind of meshes. There is no a unique mesh type that can be used as a good discretization of geometric domain to solve any problem. In this section, we will first introduce the concept of terminal-edge region and how it can be used to generate a polygon partition without overlapping inside a domain. Second, we will discuss some ideas about why we think that terminal edge regions can be an alternative concept to Voronoi regions to generate polygon meshes. Third, we are going to state several research questions and fourth the main research lines we want to explore during this project to answer the previous questions. Fourth, we are going to summarize the results of our ongoing research project and how both are related.

Inspired by our previous work to detect polygonal cosmic voids from point sets [30, 15, 31], where we have created an algorithm that starting from a Delaunay triangulation of the input point set, tessellates the domain into terminal-edge regions without overlapping, we would like to study these tessellations as a possible initial polygon/polyhedron mesh. Terminal edge regions are bounded by edges that are not the longest edge of any triangle (frontier-edges). In case of isosceles or equilateral triangles, a random order is given to the edges. Terminal edge regions can also be seen as by the union of the triangles that belong to the Longest-Edge Propagation Path of t (Lepp(t)) [32, 33] that share the same terminal-edge. The Lepp(t) is a chain of neighbour triangles where each triangle has a longest-edge greater than the longest edge of the previous triangle in the path[32]. If an edge shared by two triangles is the longest edge of both triangles it is called a terminal-edge. Figure 1 shows the terminal-edge regions found in the Delaunay triangulation of a point set. Note that terminal-edge regions are not always defined as simple polygons: some of them contains dangling edges that we have called frontier-edges (drawn in red in Figure 1). In this Figure, the partition was generated taking as geometric boundary the convex hull of a point set, but this kind of partition can be generated from any constrained Delaunay triangulation and in general, for any triangulation. This fact motivated us to formulate the following research questions: Can terminal-edge regions be adapted to be used as good basic cells for polygon numeric methods and be an alternative to Voronoi regions?. Can this approach extend to 3D and so terminal-edge regions be adapted to be good basic cells for polyhedron meshes?

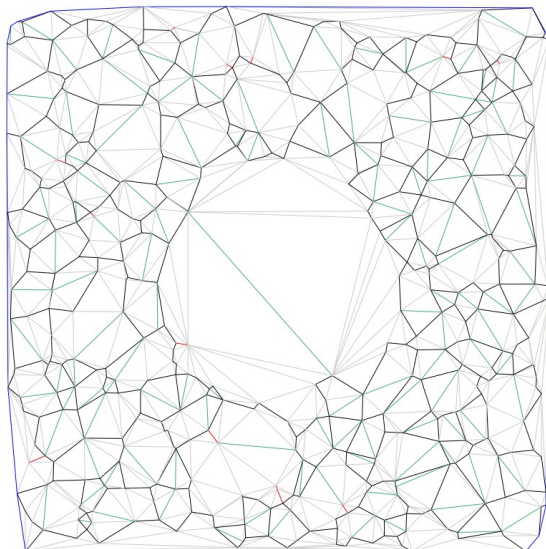


Figure 1: Terminal-edge regions in a Delaunay triangulation. The green edges are terminal-edges, the grey are internal edges, the black edges are frontier edges and barrier edges are the red edges. Each terminal-edge region is delimited by the black edges or blue edges

The concepts of terminal-edge and Lepp have already extended for tetrahedra [32, 34] so the concept of 3D terminal-edge region can be seen as the union of the tetrahedra t whose $Lepp(t)$ share the same terminal-edge. The boundary of a 3D terminal-edge region is formed by frontier and barrier faces. So from any tetrahedral mesh, we can generate a partition into terminal-edge regions. We have identified these regions in our preliminary work looking for cosmic voids [30] but without recognizing them in the way we are doing now. We believe that it is possible to generate an initial polygon/polyhedron partition from any triangulation or tetrahedral mesh. Moreover, we think that it would be possible to divide terminal-edge regions that have polygons/polyhedra with barrier-edges/barrier-faces into several simple polygons/simple polyhedra using some of the edges/faces of underlying triangle/tetrahedral mesh and so generate an initial mesh formed by simple (convex and non-convex) polygons/polyhedra.

Once we have generated an initial mesh, further research questions arise: Are polygon/polyhedron meshes based in terminal-edge regions composed of a lower number of elements, faces and edges than the Voronoi diagram with equivalent quality? Is it possible to design efficient point insertion strategies in order to adapt the mesh in real time to current simulation requirements (adaptive mesh)? Is it possible to accelerate the initial mesh generation, point insertion and optimization with gpu computing?

The algorithm we have implemented to find 2D terminal-edge regions is $O(n \ln(n))$, where n is the number of points. The computational complexity is given by the algorithm to generate the Delaunay triangulation. The edge classification part can be done in $O(n)$ and so the frontier and barrier edges of region are obtained [15]. The algorithm to generate a Voronoi diagram is $O(n \ln(n))$ so they have the same computational complexity. However Voronoi regions for points at the geometric boundary are infinite and so they have to be cut and handled in some way through the insertion of new points. This is not necessary for terminal-edges regions: they are always by construction inside the geometric domain.

Voronoi diagram based meshes have one Voronoi region (convex polygon) for each input point and the number of edges is the same as the Delaunay edges. In case of the partition based in terminal edges we have observed a strong reduction in the number of polygons and edges for the same amount of input points. From the Delaunay edges survive the ones that are frontier and barrier edges in the initial partition, this means shortest and middle edges remain. These geometric properties have brought us to formulate the previous research questions and this project. In addition, note that terminal-edges regions can be generated from any triangulation and tetrahedron meshes but we think that terminal edge regions coming from Delaunay meshes are more equilateral than terminal edge regions obtained from other triangulation of the same point set. We would like to study the properties of terminal edge regions too.

The main research line of this project will be meshing algorithms based on terminal-edge regions but since we have to compare the new algorithms with previous ones, we will need for sure to do some extensions or improvements to the tools developed in our previous work. That is why in the next part of the section, we are going to describe mainly Delanoi (Veamy) and Mesher and our ongoing work on them because they will be used as reference meshing tools. Veamy is one of the libraries to use the VEM in 2D. In concrete, we are working now on the validation, performance evaluation and determining the scope application of Delanoi (and Veamy) and of the extended Mesher.

Delanoi is a 2d meshing tool that generate constrained Voronoi based meshes and Veamy [13] is a library that implements some preliminary functionalities of the vem. The Delaunay triangulation is generated using the software Triangle [2]. The input is the geometry specified in PSLG (Planar Straight Line Graph), and a set of points inside the domain defining the desired point distribution. This input is given to Triangle and a constrained Delaunay triangulation is generated. Since Veamy is a library for the VEM, both convex and non-convex polygons can be part of the mesh. So, from the Delaunay triangulation, the Voronoi diagram was computed and cut against the boundary allowing the existence of non convex polygons at the boundary [13]. Both tools are available from <https://cemcen.cl/>.

The extended Mesher has been built by integrating an algorithm to tessellate any convex polyhedron with Steiner points (1-irregular elements) into a Delaunay tessellation [11]. Mesher is a mixed element mesh generator based in the standard octree approach [35, 36], developed by Lobos et al. [12]. This mesh generator first generates a mesh that fulfil the point density requirements by dividing each octant into

eight smaller octants, then generates an 1-irregular mesh, and finally approximates the domain geometry and tessellates the 1-irregular elements using a pattern-based approach. Instead of this pattern-based solution, we use now an algorithm that connect the points using the largest co-spherical element as possible, and so reduces the number of points, edges, faces and elements in the final mesh in comparison with the original Mesher. This approach use a unique refinement approach, but can model any domain geometry specified by one or several closed 3D (surface) triangulations. We have decided to concentrate our efforts in developing a scalable and efficient polyhedron meshing tool as an option of Mesher instead of improving prototype polyhedron meshing algorithm based on modified octrees [23, 11]. The main reason is that Mesher is very robust, efficient and can generate meshes for very complex geometric domains. It can handle several surface triangulations defining the input geometry. Since the VEM accepts any polyhedron, we plan to improve the original Mesher representing the domain geometry as it is specified in the input. Boundary polyhedra will be formed by faces of the input surfaces and the cut octant. Moreover, the good characteristics of the mesh generator [11], we will be integrated into Mesher. One of these is a flexible refinement approach into 2,4 or 8 octants depending on the required refinement direction.

Related work and state of the art methods

The most used 2D meshes for standard finite element methods are triangulations [37, 2, 1] or quadrilateral meshes [38, 39, 40]. 2D Mixed meshes composed of triangles and quadrilaterals have been also used but are not so common [41] as the previous ones. The most used 3D meshes are composed by tetrahedra [35, 36, 42, 43, 3] or hexahedra [44, 45, 46]. There are also some approaches to generate mixed element meshes [47, 23] and polygon and polyhedron meshes [48, 25, 6]. In particular, Garimella et al. [48] proposed a method for the generation of polyhedron meshes based on the dual of a general tetrahedral mesh that can handle non-manifold boundaries. They also propose quality criteria for the evaluation of the quality of the generated polyhedra. Other methods that use the Voronoi diagram as the polygon/polyhedron mesh, and then the Voronoi cells as mesh elements are [25, 26, 27].

The generalization of finite element methods to include polygons/polyhedra as part of the mesh elements was started in the last decade [4, 5, 49]. Polygonal elements are usually generated by using a quadtree approach and from Voronoi cells. Since the introduction of the VEM seven years ago [16, 17], several research groups have been developing computational frameworks for using the VEM, both in 2D and 3D, in order to explore how far the vem can be used to solve new problems. To mention some examples, the VEM has been formulated and applied to solve linear elastic and inelastic solid mechanics problems [50], in fluid mechanics [51], in the optimization of a fluid problem through a discrete network [28], for compressible and incompressible nonlinear elasticity [52], for finite elasto-plastic deformations [53] and brittle crack propagation [19].

Several parallel approaches have been already published in order to accelerate the different steps of the meshing generation process for multi-core and gpu architectures. We are not going to be exhaustively, but we are going to mention some of them. On gpu, novel 2D parallel algorithms for the generation of Delaunay meshes are [54, 55, 56, 57]. To our knowlegde efficient algorithms for parallel point insertion using gpu computing as needed by the refinement algorithms or to improve the quality of some elements have not been published yet, but several works on multicore and cpu clusters are available. With respect to tetrahedral mesh refinement, Antonopoulos et al. [58] proposed a parallel algorithm that refines a mesh on a parallel multilevel architecture with distributed memory. Rivara et al. [59] proposed a global parallel refinement for tetrahedral meshes based on the terminal edge refinement concept on a multi-core architecture. [60] presents an study of a multi-thread lepp-bisection algorithm for mesh refinement. For mesh improvement, [61] shows a parallel algorithm to untangle and to improve the quality of a mesh for distributed-memory machines and [62] proposes a parallel algorithm for implementing the edge-collapse strategy on gpu architectures. We hope to find some open-source parallel mesh generators in order to be able to compare our approaches.

Hypotheses

Our working hypotheses in the context of numerical methods that benefit from meshes composed of as less polygons/polyhedra as possible but respecting the point density requirements are:

- Terminal-edge regions can be transformed into simple polygons/polyhedra (convex or non convex) and the generated partition can be used as the starting point of an initial polygon/polyhedral mesh
- Refinement and optimization strategies can be extended to improve the quality polygons/polyhedra according to the point density and quality shape metrics required by the problem to be solved and the used numerical method
- Polygon/polyhedron meshes based in terminal-edge regions will contain less points, edges and cells than meshes based in the Voronoi diagram for the same required point density and similar accuracy in the simulations results

We expect not only to reduce the number of edges, faces and cells, but also the number of points because our algorithm would not need to add points at the boundary to represent the domain geometry as the constrained Voronoi mesh generators have to do while cutting Voronoi regions. Moreover, during the mesh refinement and improvement steps, the algorithm should always consider the possibility of generating non-convex polygons if they are adequate and so keep the number of inserted points as low as possible. We want to explore how strong can be the performance improvement in both storage and computational time while generating the mesh and solving specific problems with the VEM.

General and specific goals

The main goal of this project is to propose novel strategies to generate polygon/polyhedron meshes that can be useful for solving scientific and engineering problems with numerical methods that benefit from these kind of meshes.

The specific goals planned to reach the general goal are:

1. Study the geometric properties of polygons/polyhedra generated from terminal-edge regions from both Delaunay and non-Delaunay partitions.
2. Identify and develop quality metrics for polygons/polyhedra to guide the refinement and optimization processes of a mesh
3. Develop an algorithm to generate an initial polygon/polyhedron mesh from terminal-edge regions
4. Design or extend current refinement optimization strategies to fulfil the point density requirements imposed by the used numerical method and the problem itself.
5. Compare the generated meshes with Voronoi based meshes in terms of number of points/elements, accuracy of the simulation results and simulation time/memory, among other comparison criteria.
6. Explore the design of gpu based algorithms to accelerate mesh generation and the evaluation processes for 2D polygon meshes

Methodology

We have planned to execute this project in three years. The chosen programming language chosen is c++ because we can program very efficiently and at the same time use the object oriented paradigm when is appropriate. For parallel programming we have experience with cuda and opencv. The advantage of opencv is that we can use any video card while cuda can be only used with NVidia cards. As design and programming methodology we are using an object oriented approach which has allowed us to build software easy to extend, adapt and reuse. We put emphasis in building reusable and extensible components,

where efficiency is not sacrificed. In order to reach the general and specific goals described above, we plan to do the following:

1. Study the geometric properties of polygons/polyhedra generated from terminal-edge regions from both Delaunay and non-Delaunay partitions.

As Delaunay triangulations produce more close to equilateral triangles than any other triangulation, we have the intuition that terminal-edge regions based on them should have a "better" shape. In order to fulfil this objective, we plan to carry out a theoretical and empirical study in both 2D and 3D, by studying the minimum and maximum internal angles, number of sides, convexity index among other geometric properties.

2. Identify and develop quality metrics for polygonal/polyhedral elements to guide the refinement and optimization processes of a mesh

There are some quality metrics for polygon elements, such as convexity index, aspect ratio and area; and for polyhedron meshes, like volume, size of the faces and convexity index, among others. It seems that for the VEM there is no requirement on the polygon/polyhedron shape and for the pfem, convex elements are preferred depending on the problem to be solved but we have to explore and design quality metrics for the transition between coarse and fine regions. For both methods an adequate point distribution is key. Therefore it is important to identify and implement criteria to guide the point insertion and optimization of the mesh. We are going to use as a reference the study done by Attene et al. [63] where the authors select several geometric properties of polygonal elements and evaluate the performance of solvers such as the VEM in order to obtain a correlation among them.

3. Develop an algorithm to generate an initial polygon/polyhedron mesh from terminal-edge regions

Starting from the terminal-edge region built by our cosmic void searching algorithm, we can already count with an initial mesh composed by simple polygons after eliminating barrier edges. We are now using Detri2 instead of qhull (www.qhull.org) because we want to generate meshes for any kind of geometry specified in the PSLG format. With qhull we did it only inside the convex hull of a point set. With Detri2 we can generate a constrained Delaunay triangulation and from this triangulation edges are classified as terminal-edges, barrier edges, frontier edges and internal edges. This is done in $O(n)$. Later, polygons are build clock-wise by running through barrier and frontier edges. Barrier edges are either eliminated or mark as internal segments to be used to divide the non-simple polygons into simple ones. This algorithm has to be extended to 3D to generate the initial polyhedron mesh. We are going to start using Tetgen to generate the constrained Delaunay tetrahedral mesh, and then label terminal-edges, barrier edges and faces in order to build the polyhedra. The use of Tetgen will allow us to handle and model very complex domain geometries.

4. Design or extend current refinement optimization strategies to fulfil the point density requirements imposed by the used numerical method and the problem itself.

We have to identify first in which direction or region more points and/or smaller cells are required. We will try to avoid cell splitting if we can just insert points on existing edges. The refinement will be very flexible, allowing to use non-convex cells and minimizing the number elements while keeping the desired quality. In 2D, this means to divide an element into two polygons and in 3D, into several polyhedra. For example, if an element is not completely cut by a fracture, it is possible that only interior points are required to form a non-convex polygon[19]. When no refinement direction is specified, elements will be partitioned at edge-midpoints or at points previously used/inserted. We plan to keep the underlying triangle/tetrahedral mesh because this can guide the refinement/point insertion operations. We plan to use triangle edges or tetrahedron faces of the Delaunay mesh as initial candidates to be inserted.

5. Compare the generated meshes with Voronoi based meshes in terms of number of points/elements, accuracy of the simulation results and simulation time/memory, among other comparison criteria.

We have already developed the visualization tool called Camarón [64] for the inspection of polygon and polyhedron meshes, manifold and non-manifold geometries. Several geometric quality criteria for both

surface and volume meshes are already implemented, and also contour lines and isosurfaces can be built from scalar simulation data. We plan to improve its scalability and to add new functionalities that allow us to visualize the geometrical properties together with simulation data computed at the mesh and integration points. We believe that these functionalities will allow us to understand unexpected results. We also plan to compare the new meshes with Voronoi based meshes generated our tools (Delanoi and Veamy) and Mesher, but also with the meshes generated by Polymesher or the computational framework described in [19]. We also plan to validate the polygon/polyhedron meshes by modeling problems already solved by the VEM in in terms of accuracy and computational time. In the last years, several simulation results has been published and we can use these as a reference to validate our meshes [19, 13].

6. Explore the design of gpu based algorithms to accelerate mesh generation and evaluation processes for 2D polygon meshes

We have already explored the possibility to parallelize the edge classification step while identifying terminal edge regions [31] and we think there is a possibility to go further in particular to explore how cells can be divided in parallel using gpu computing. Gpus provide an efficient architecture to solve data parallel problems and when there is no need to ask for memory dynamically. This last aspect has been a problem to design algorithms for inserting points in parallel. One of the co-researchers is interested in designing a special kind of buffers, whose preliminary data structure is based on hash functions, in order to deal with this problem. We have already implemented meshing algorithms on gpu [57] and developing mapping functions [65, 66] that allows the solution to use all the available threads. That is why we think this is a proper way to start. We will evaluate the impact of the parallel strategies against the sequential version. In particular, to determine from which mesh size parallel gpu-computing is useful.

Work Plan

Depending on the interest of the students, some activities described in next table can be interchanged. Each year, we plan to send at least one paper to an international journal (Engineering with computers, CAD journal, International journal of numerical Methods in Engineering and numerical algorithms, among others) and at least one paper to an international conference (International Meshing Roundtable IMR and International Conference in Computational Sciences ICCS), among others)

Specific Goals	Year 1	Year 2	Year 3
Study the geometric properties of polygons and polyhedra generated from terminal-edge regions in both Delaunay and non-Delaunay partitions. Characterize them.	Theoretical and empirical study in polygons by studying the minimum and maximum internal angles, number of sides, convexity index among other geometric properties. Compare with Voronoi polygons.	Theoretical and empirical study in polyhedra by studying the minimum and maximum dihedral angles, number of sides, convexity index among other geometric properties. Compare with Voronoi polyhedra	Revision of the results in terms of new challenges or new possible applications
Identify and develop quality metrics for polygons/polyhedra to guide the refinement and optimization processes of a mesh	Identify metrics in 2D to increase the point density as required for the VEM and to guide the optimization step. Implement these algorithms	Validation and testing of the implemented algorithms.	Identify metrics in 3D, to increase the point density as required for the VEM and to guide the optimization step. Implement these algorithms
Develop an algorithm to generate an initial polygon/polyhedron mesh from terminal-edge regions	Develop an algorithm to generate an initial polygon mesh from terminal-edge regions obtained from Delaunay	Develop an algorithm to generate an initial polyhedron mesh from terminal-edge regions obtained from Delaunay	Study the scope of application of these two meshing algorithms.

	triangulations generated by Detri2. Study its performance in terms of memory use and time.	meshes generated by TetGen.	
Design or extend current refinement optimization strategies to fulfil the point density requirements imposed by the used numerical method and the problem itself.	As in Delanoi, include density function to insert a priori points inside the 2D domain (these would appear in the initial mesh) and design a refinement approach to fulfil the quality metrics	Implement density functions to insert a priori points inside the 3D domain (these would appear in the initial mesh). Implement strategies to refine elements if necessary	Design and implement a refinement approach to fulfil the requirements of the quality metrics as far as possible.
Compare the generated meshes with Voronoi based meshes in terms of number of points/elements, accuracy of the simulation results and simulation time/memory among other comparison criteria.	Start the validation of the generated 2D polygon meshes in simple problems already solved using Delanoi and Veamy.	Evaluation of the computational performance of the 2D polygon meshing algorithm; comparison with meshes based in the Voronoi diagram as the ones generated by Delanoi.	Evaluation of the 3D polyhedron meshing algorithm; comparison with meshes based in the Voronoi diagram in terms of number of points/elements, accuracy of the simulation results and simulation time/memory
Explore the design of gpu based algorithms to accelerate mesh generation and evaluation processes for 2D polygon meshes	Explore dynamic parallel point insertion algorithms for 2d mesh generation. Identify part of the 2D meshing algorithms that can be transformed into data parallel problems and implement them	Explore dynamic parallel point insertion algorithms for 3d mesh generation. Identify part of the 2D meshing algorithms that can be transformed into data parallel problems and implement them	Explore dynamic parallel point,edge, face insertion algorithms for 2d mesh generation. In particular for the basic flipping operation in 3D in a tetrahedron mesh

Work in progress

As stated in the formulation of this project, this is a continuation of our current Fondecyt project No 1181506, which finishes in March 2021. For the formulation of the VEM to solve new problems, we are collaborating with Dr. Alejandro Ortiz- Bernardin from the Mechanical Engineering Department at the University of Chile [13, 67]. We count with a first version of our own 2D open source 2D meshing tool to generate constrained Voronoi diagrams (Delanoi) which will be used at the beginning as reference tool to compare the 2D meshing algorithm proposed in this project. We also count with the VEM library (Veamy) [13] to evaluate and compare the generated meshes by solving a problem. As starting point for the meshing algorithms based in terminal-edge regions we count with the algorithm that computes terminal-edge regions inside 2D and 3D Delaunay triangulations inside the convex hull of a point set [15, 30]. In addition, the student Sergio Salinas is starting a PhD (granted by the Chilean science foundation Anid) in this topic at the Computer Science Department under my supervision. He has already successfully done preliminary tests in order to generate a polygon partition based in terminal-edge regions from constrained Delaunay triangulation generated by Detri2. As part of my current Fondecyt project, we are also testing the integration of the polyhedron tessellation algorithm [11] in Mesher and determining for which kind of problems these meshes can be used with the vem. The extended Mesher will also be used as a reference tool. We also count with a visualization tool called Camarón [64] to analyse and visualize polygon/polyhedron meshes.